Bitmap Compression: Formats and Experiments

ATDA · Assignment 1 companion

This handout explains **PLWAH, CONCISE, VAL-WAH, SBH and BBC**. Use the section for your group's additional format alongside the Raw, WAH, EWAH and Roaring representations covered in Lecture 3. You do not need to read research papers to understand the explanations or carry out the suggested experiments. References at the end are optional.

How to use the handout

Each format has two pages: a worked encoding, followed by an AND example and research questions. The questions suggest experiments; they are not claims about which format will win. You can give an ordinary presentation using your own examples and measurements.

Section	Main idea	Pages
PLWAH	Attach an almost uniform group after a fill	2–3
CONCISE	Allow an exception in the first group of a fill	4–5
VAL-WAH	Choose the segment length for each bitmap	6–7
SBH	Encode small groups; process bounded regions	8–9
BBC	Encode a fill and a tail of bytes together	10–11
Experiment guide / optional references	Shared measurement rules / sources	12 / 13

Notation used throughout

N is the logical bitmap length. Positions are numbered from 0. **k** is the number of set bits; density is k/N. For inputs A and B, **t = |A ∩ B|** is the number of shared set positions. AND produces A ∩ B; OR produces A ∪ B.

A **group** or **segment** is a fixed number of consecutive original bits. A **literal** stores these bits explicitly. A **fill** stores repeated all-zero or all-one groups using a counter. A 32-bit encoded word is not the same thing as a 31-bit original group.

In logical input strings, positions increase **from left to right**. Encoded fields are printed most-significant bit first; their position fields can use a different numbering rule, stated in the relevant section. A vertical bar separates fields or groups and uses no storage. Z means an all-zero group; O means an all-one group.

Size convention. Worked counts include only the fields explicitly shown. They exclude external length metadata, object overhead and file framing unless stated. Actual saved sizes must include the metadata your implementation writes. A packed raw bitmap needs ceil(N/8) bytes; a word-based raw implementation may round up further.

1. PLWAH

Position List Word Aligned Hybrid · encoding

The idea

A single different bit can force WAH to store an extra literal. PLWAH reserves space in a fill word for positions of exceptional bits in the **following** group. That group can then be reconstructed without storing a separate literal. Here we use the paper's 32-bit, one-exception configuration. [1]

Fields in this configuration

Input groups contain 31 bits. A literal is `0 | 31 payload bits`. A fill is:

```
1 | v | p (5 bits) | c (25 bits)
```

Here **v** is the fill bit and **c** counts uniform groups. If **p = 0**, there is no following exceptional group. Otherwise append one more group of v bits and flip its p-th bit, numbered **1–31 from the left**. Thus p ≠ 0 adds a group beyond the c uniform groups. Other PLWAH configurations can reserve more position fields. [1]

Worked example: 124 original bits

Let Z contain 31 zeros. Let L contain a one in its first position and 30 zeros.

```
Original groups:   Z | Z | Z | L
L:                 10000000000000000000000000000000
PLWAH fields:      1 | 0 | 00001 | 00000000000000000000000011
Meaning:          zero fill; p = 1; c = 3
```

Decoding is concrete: write 3 × 31 = 93 zeros, then append L. The result is 124 bits with its only one at global position 93. The counter is **3**, not 4: the exceptional group follows the counted fill.

Representation	Encoded content	Bytes
PLWAH-32, one exception	One fill word with a position field	4
WAH-32	One zero-fill word + one literal word	8
Packed raw	ceil(124/8)	16

All rows represent the same 124 logical positions. These are calculated payload sizes, not library measurements. The original bits are not shortened to make the example; 31-bit groups and a 32-bit codeword are used.

A nearby case

Change L to `11000000000000000000000000000000`. A single position field cannot describe both ones, so this configuration needs a separate literal: 8 bytes in total. The useful property is not simply low density; it is the placement of at most one exceptional bit in the group immediately after a fill.

PLWAH: operations and experiments

Example operation: AND

Follow the logical ranges

Use the A from the previous page. Let B contain two all-one groups, one all-zero group, then a literal with ones at its first and last positions.

Original groups	A	B	A AND B
1–2	Z, Z	O, O	Z, Z
3	Z	Z	Z
4	100...000	100...001	100...000

The output again has a zero fill of three groups followed by a group with one exceptional bit. It can use the same 4-byte PLWAH word as A.

An iterator must expose the uniform part of a fill **before** its exceptional trailing group. For this example it first handles 93 zero bits, then reconstructs the last 31-bit group. Encoded words themselves are control information: ANDing their 32-bit integer values would not compute the bitmap intersection. [1]

Research question

When does the position field save enough words to improve both size and query time?

Experiment A: eligible versus ineligible groups. Build repeated regions containing a zero fill followed by one mixed group. Compare mixed groups containing one one with mixed groups containing two ones. Keep the bitmap length fixed. To isolate placement from cardinality, also construct paired inputs with identical N and k but distribute the ones differently among the mixed groups. Record how many groups can be attached to fills.

Experiment B: order matters. Compare repeated patterns Z | Z | Z | L and L | Z | Z | Z. Repeat enough times for a stable measurement and count the actual encoded words: the end of one repeated pattern may interact with the start of the next. Use the same N and k.

For both experiments, compare PLWAH with WAH, EWAH, Roaring and Raw. Measure size, construction time, and AND/OR time on input pairs with recorded overlap. Explain whether the extra position decoding offsets the saving in words read.

What to show in your explanation

Show one word that absorbs a trailing group and one nearby case that cannot. Decode both back to the original set positions. Relate the measured size to the number of fills, remaining literals and attached groups; do not infer running time from byte counts alone.

Implementation boundary. The examples specify one exception per fill and omit the paper's adaptive counter extension. If you implement this subset, label it **PLWAH-32, one exception, fixed counter**. Split fills that exceed the 25-bit counter. Keep N externally so final padding never becomes part of the logical set.

2. CONCISE

Compressed ’n’ Composable Integer Set · encoding

The idea

CONCISE can put an exceptional bit in the **first** group of a fill. This differs from PLWAH, where the exceptional group follows the uniform groups. The original 32-bit CONCISE layout also uses the opposite literal/fill flag convention to our lecture's WAH. [2]

Fields in the original layout

Each original group has 31 bits. A literal is `1 | 31 payload bits`. A fill is:

```
0 | v | p (5 bits) | q_minus_1 (25 bits)
```

The word covers **q = q_minus_1 + 1** groups of v bits. If p = 0, all are uniform. Otherwise flip one bit in the first group: p = 1 means the **rightmost** bit; p = 31 means the leftmost. The exceptional group is included in q. [2]

Worked example: 124 original bits

Let Z contain 31 zeros. Let L contain 30 zeros followed by one one.

```
Original groups:   L | Z | Z | Z
L:                 00000000000000000000000000001
CONCISE fields:    0 | 0 | 00001 | 00000000000000000000000011
Meaning:          zero fill; p = 1; q_minus_1 = 3
```

Write q = 4 groups of zeros, then flip the rightmost bit of the first group. Our logical display numbers positions left to right, so this is global position 30. This is consistent with the field's right-to-left bit numbering; the two conventions describe different things.

Representation	Encoded content	Bytes
CONCISE-32	One fill with an exception in its first group	4
WAH-32	One literal + one zero-fill word	8
Packed raw	ceil(124/8)	16

The same 25-bit value **3** appeared in the PLWAH example, but it has a different meaning. There it counted three uniform groups before an extra exceptional group. Here it means four groups in total, including the exceptional first group.

A nearby case

If the first group has two ones, it cannot be described by a single flipped position. Store that group as a literal, followed by a pure zero fill for the other three groups: 8 payload bytes. This lets you test the mechanism with a one-bit change to the input.

CONCISE: operations and experiments

Example operation: AND

Reconstruct the exceptional first group

Let A be the previous example. Let B's first group have ones in its two rightmost positions, followed by three all-one groups.

Original groups	A	B	A AND B
1	000…001	000…011	000…001
2–4	Z, Z, Z	O, O, O	Z, Z, Z

The result contains only global position 30. It again fits in the one CONCISE word shown on the previous page. If B's first group instead ends in `…010`, the intersection has no set positions.

For a modified fill, an iterator exposes the exceptional **first** group and then the remaining uniform groups. It can process those pieces against B's logical ranges without expanding the complete bitmap. Output must then be encoded again. [2]

Research question

How much do exception placement and alignment explain the difference between CONCISE, PLWAH and WAH?

Experiment A: move the same set bits. Construct long bitmaps with fixed N and k. Shift a pattern through all 31 offsets relative to the group boundary, keeping the shifted pattern inside the universe so no bits are lost. Measure encoded size and count mixed groups. Compare CONCISE with WAH, EWAH, Roaring and Raw; add PLWAH if available in your framework.

Experiment B: one versus several exceptions. Make inputs where mixed groups usually contain one one, then inputs where the same total number of ones is concentrated into fewer mixed groups. Keep N and k fixed. Count the groups eligible for CONCISE's exception encoding and test whether this predicts the size ranking.

Use a separate set of paired inputs to measure AND/OR. Record the intersection size t rather than assuming that equally dense inputs share many positions. Inspect whether a smaller encoding is also faster for these pairs.

What to show in your explanation

Show the flag convention, the meaning of $q - 1$, and the location of the exceptional bit. Give one complete decoding and one AND example using your own inputs. Include at least one data pattern on which the exception mechanism offers no saving over a literal.

Implementation boundary. CONCISE's 25-bit group-count field limits a single fill's span. Use the representable universe of the implementation you select and record it. Handle lengths not divisible by 31 using external N and zero padding. The bit-position convention must be consistent in encoding, decoding and queries.

3. VAL-WAH

Variable Aligned Length framework with WAH · encoding

The idea

WAH normally uses one group width throughout a representation. VAL-WAH selects a segment length **for each bitmap** from a compatible family. The paper's 64-bit configuration uses s ∈ {15, 30, 60}. Segment lengths are multiples of one another, allowing conversion during queries. [3]

Storage in the illustrated configuration

Each compressed segment is a literal or a fill. Its type flag is moved to a header at the front of the 64-bit word. A fill payload contains a value bit and an (s − 1)-bit segment counter. For s = 15, four 15-bit payloads and four flags fill one word. The format also has a one-byte bitmap selector identifying the encoding and segment length. [3]

```
s = 15:   4 flags | payload 1 | payload 2 | payload 3 | payload 4
            4 b       15 b        15 b        15 b        15 b
```

Worked example: 105 original bits

Define 15-bit groups Z = all zeros, O = all ones, L_1 = `100000000000001`, and L_2 = `010101010101010`.

```
Original:    Z | Z | L₁ | O | O | O | L₂
s = 15:     F(0,2) | L₁ | F(1,3) | L₂
Word flags: 1010        (1 = fill, 0 = literal)
```

F(v,c) means a fill of c segments of value v. It is a label for the fields, not extra stored text. There are four encoded segments, so all fit in one 64-bit word. Include the one-byte selector: **8 + 1 = 9 bytes**, excluding external N and other framing.

Segment length	Groups after zero padding	Encoded segments	Storage with selector
15	7 groups	F(0,2), L_1, F(1,3), L_2	1 word + 1 B = 9 B
30	4 groups	F(0,1), literal, F(1,1), literal	2 words + 1 B = 17 B
60	2 groups	Two literals	2 words + 1 B = 17 B

For s = 30, the groups are ZZ, L_1O, OO and L_2Z; the last Z is padding. For s = 60, each group mixes zeros and ones. Packed raw uses ceil(105/8) = 14 bytes. Counts assume full stored 64-bit words, including unused slots or pad bits. They illustrate this input only.

VAL-WAH: operations and experiments

Example operation: AND across different segment lengths

Align logical positions before combining them

Let A use 15-bit segments and B use 30-bit segments. Consider the first 30 logical positions:

```
A:     Z              | 100000000000001
B:     111111111111111  100000000000000
AND:   000000000000000  100000000000000
```

B's 30-bit literal can be viewed as two consecutive 15-bit pieces. The first meets A's zero segment and contributes zeros. The second meets A's literal and retains only its leftmost one. The result has one set bit at global position 15.

Similarly, a fill of c 30-bit segments spans 2c 15-bit segments. The iterator can convert the current piece while processing the query; it need not rewrite the complete inputs to a common stored format. The paper's tuning parameter λ trades compression against query-processing cost. [3]

Research question

Does the segment length that minimizes storage also minimize AND/OR time?

Experiment A: separate the choices. For identical logical inputs, force s = 15, 30 and 60. Use random data, long uniform runs and short runs interrupted by mixed segments. Measure size, construction time and AND/OR time. Record partial final words and zero padding so short inputs do not give a misleading size comparison.

Experiment B: different lengths in the two inputs. Hold A and B fixed and compare (15,15), (30,30), (60,60), (15,30) and (15,60) as their encoded segment lengths. Record t and the output representation. Explain any cost from converting segment lengths rather than attributing all differences to compression.

If the implementation exposes λ, compare its automatic choice with the three forced choices. Treat selector profiling time as part of construction, or report it separately. Do not pick the best setting using the same timed query and then omit the cost of that selection.

What to show in your explanation

Explain the difference between a physical 64-bit word and a logical segment. Show how one physical word holds multiple encoded segments and trace one operation across unequal segment lengths. Use a measured plot of size against query time to justify a configuration choice.

Implementation boundary. Testing WAH with different fixed widths alone does not reproduce VAL-WAH: the packed flag header and processing across segment lengths are part of the design. A reduced experiment can use forced legal lengths instead of implementing the automatic selector; label that scope explicitly. Physical word padding and the selector still count toward saved size.

4. SBH

Super Byte-aligned Hybrid · encoding

The idea

SBH encodes 7-bit groups into bytes and partitions the bitmap into larger regions called **super-buckets**. A super-bucket contains a configured number of groups. It bounds the region expanded for query processing; it is not a Roaring container with a choice of representations. [4, 5]

Byte rules

A literal is `0 | 7 payload bits`. A zero fill starts with `10`; a one fill starts with `11`. The remaining six bits store a counter digit. A longer counter uses successive bytes with the same fill prefix, least-significant six-bit digit first. Counters restart at super-bucket boundaries; fills do not cross those boundaries. [5]

Worked example: one 56-bit super-bucket

Choose eight 7-bit groups per super-bucket. Let Z = `0000000`, O = `1111111`, L_1 = `1010101`, and L_2 = `0110011`.

```
Original:     Z | Z | Z | L₁ | O | O | L₂ | Z
```

Original groups	Encoded byte	Interpretation
1–3: Z Z Z	10 000011	Three zero groups
4: L_1	0 1010101	Literal
5–6: O O	11 000010	Two one groups
7: L_2	0 0110011	Literal
8: Z	10 000001	One zero group

These are **5 encoded bytes for 56 original bits**, compared with 7 bytes for packed raw. The count excludes storage for N, the configured super-bucket size and any external framing.

Why successive fill bytes are not separate runs

In a sufficiently large super-bucket, 70 zero groups use these two bytes:

```
10 000110   10 000001
low digit 6; high digit 1:   6 + 64 × 1 = 70 groups
```

They do not mean 6 + 1 groups. At the known super-bucket boundary, reset the counter interpretation. For example, two adjacent all-zero super-buckets of eight groups each use two independent `10 001000` bytes. A decoder knows that each describes eight groups in a different region. All examples use the published counter rules, not the lecture's fixed six-bit WAH fill counter.

SBH: operations and experiments

Example operation: AND using a bounded buffer

Expand one region, combine, continue

Use the preceding 56-bit A. Let B have all-one groups except for group 4, which is `0011001`. After expanding the corresponding super-buckets into working buffers, combine matching bits:

Groups	A	B	A AND B
1–3	Z Z Z	O O O	Z Z Z
4	1010101	0011001	0010001
5–6	O O	O O	O O
7	0110011	O	0110011
8	Z	O	Z

The resulting groups can be re-encoded into five bytes: `10 000011`, `0 0010001`, `11 000010`, `0 0110011`, `10 000001`.

SBH uses super-bucket expansion to support query processing. A zero-initialized buffer can advance over a zero fill without writing every zero individually. Buffer expansion is still work and must be included when timing a query. [4, 5]

Here, a clear way to materialize AND is: decode A's current region, decode B's matching region, AND the buffers, and encode the result. This is a correct baseline procedure, not a claim to reproduce every optimization in the authors' implementation. Process the next region after the current output is consumed.

Research question

How does super-bucket size affect compression, temporary memory and query time?

Experiment A: vary the region size. Choose several supported lengths, for example 56, 224 and 896 original bits. Keep the logical inputs identical. Measure size, construction and AND/OR time, and report the working-buffer size. Count fills split by region boundaries.

Experiment B: boundary placement. Move the same long zero/one runs relative to region boundaries without changing N or k. Compare these inputs with scattered mixed groups. Explain whether more split fills or more buffer work accounts for the result. Also inspect inputs with almost no fills.

What to show in your explanation

Decode a literal, a one-byte fill and a two-byte counter. Show where a super-bucket ends. Trace the query's buffer contents and identify whether reported time includes re-encoding the result.

Implementation boundary. Do not call SBH simply “8-bit WAH”: its counter continuation and bounded expansion matter. If you use the baseline procedure above, label it as such. Compare materialized compressed outputs consistently, or explicitly report a separate experiment with raw-buffer output. Verify fills of 63, 64 and 70 groups and fills touching a region boundary.

5. BBC

Byte-aligned Bitmap Compression / Code · encoding

The idea

BBC groups the original bitmap into **8-bit bytes**. A compressed unit describes a fill of uniform bytes followed by a tail of explicit bytes. A control byte tells the decoder what follows. Special cases also encode a byte with a single exceptional bit. [6]

BBC has variants. For a concrete byte example, we use the short-fill control layout in Antoshenkov's patent [6]; this is not a claim that every BBC library uses identical flags.

Short fill followed by explicit bytes

For this case, the control byte has three fields:

```
g (3 bits) | v (1 bit) | n (4 bits)
```

Here g ∈ {0,1,2,3} is the number of fill bytes, v is their bit value, and n is the number of following explicit bytes. The example uses n > 0; n = 0 has special meanings. Longer fills and exceptional-byte cases use other control layouts. [6]

Worked example: 32 original bits

```
Original bytes:  00000000 | 00000000 | 10100101 | 01011010
Control fields: 010 | 0 | 0010
Stored bytes:   01000010 | 10100101 | 01011010
```

Read the control byte as “write two zero bytes, then copy the next two bytes.” It produces 16 zeros followed by the two explicit bytes, exactly reconstructing the original 32 bits.

Item	Stored bytes
Control byte	1
Two explicit tail bytes	2
Total encoded unit	3
Original packed bitmap	4

This count excludes an end marker and external framing. In the patent's terminated stream, append the zero control byte: **4 bytes including that terminator** for this one-unit example. Always state which boundary you measure.

What is different from WAH and EWAH?

The original grouping unit is eight bits, not WAH-32's 31 or EWAH-64's 64. One control byte can describe a fill and a following literal tail. Unlike the fixed 64-bit marker shown for EWAH, BBC selects among control cases and can use extra bytes for long counts. [6]

BBC: operations and experiments

Example operation: AND by logical byte ranges

Match the bytes described by the controls

Use the preceding 32-bit A and let B consist of four bytes of `11110000`.

Original bytes	A	B	A AND B
1–2	00000000 each	11110000 each	00000000 each
3	10100101	11110000	10100000
4	01011010	11110000	01010000

The zero fill determines the first two output bytes without consulting their individual bits. Apply ordinary byte AND to the two tail bytes. For this result, the same short control applies:

```
01000010 | 10100000 | 01010000
```

This is three encoded bytes, or four with the patent's terminator. A control parser must keep the fill length, tail length and current logical position; it cannot AND the encoded byte streams directly. [6]

Research question

When does byte-level grouping reduce storage enough to compensate for control parsing during queries?

Experiment A: choose the scale of the runs. Generate runs that are long compared with 8 bits but short compared with 31 or 64 bits, and then generate much longer runs. Keep N and k matched between comparison cases. Compare BBC with Raw, WAH, EWAH and Roaring on size and AND/OR time.

Experiment B: shift the pattern. For a pattern placed safely inside a larger universe, test shifts by 0–7 positions to change its byte alignment. Keep N and k fixed. Count uniform bytes, explicit tail bytes and control bytes, then explain the changes in saved size.

Experiment C: amortize control information. Compare many short tails with fewer longer tails, keeping total N and k fixed. Can BBC share one control over enough explicit bytes to offset its parsing cost? Use the actual implementation's allowed tail lengths, and count its exceptional-byte cases if supported.

What to show in your explanation

Show where each control byte ends and where its tail begins. Decode one whole unit, calculate its size and trace AND over a fill and two tail bytes. State the exact BBC variant and include terminators and length metadata in measured saved sizes.

Implementation boundary. The preceding page defines one valid control case, not a full BBC encoder. A decoder for this case alone is limited to inputs represented by such units; arbitrary inputs require additional control cases. Label it a **restricted BBC subset using the patent layout**. Interpret its timings within that scope. An agreed full implementation can be used without reading the patent.

From a format to an experiment

Shared guide for all five topics

Explain a mechanism, then test a prediction

Pick a research question from your format's section. Before timing, state an expected effect and why the encoding should cause it. Use a small example to check the explanation, then generate larger inputs to test the prediction. A result that contradicts your prediction is useful if you investigate it.

Keep comparable	Record explicitly
Same logical inputs for every format	N, k, seed, value pattern and run settings
Same A and B for every queried format	a = size of A, b = size of B, t = size of their intersection
Same query output requirement	Materialized result, its format and its cardinality
Same measurement boundary	Optimization, allocation, decoding and output encoding included or excluded

Measurements and correctness

Saved size: bytes in the saved representation, including the metadata it actually requires. Keep calculated payload size separate. **Construction:** build from the same prepared positions; specify whether optimization is included. **AND/OR:** include all work required to produce the stated output, including buffer expansion or re-encoding when needed.

Generate inputs and verify outputs outside the timed interval. Check decoded positions against Raw, not just the number of ones. Repeat timings, warm up where needed, and report a summary such as the median with a measure of variation and the number of runs. Record hardware, library versions, settings and word widths.

Useful checks include no set positions, all positions set, a single set position, a partial final group, identical and disjoint inputs, and a fill that reaches a counter or region boundary. For overlap experiments, feasible t satisfies $\max(0, a + b - N) \le t \le \min(a,b)$.

Presenting the investigation

An ordinary presentation can explain the format, decode one original example, trace AND and discuss measured results. Include the generator and settings needed to reproduce the plots. Explain both a favourable case and a case where the format's special mechanism is less useful.

Scope of implementations. These pages support understanding and experimental design; they do not define every production-format corner case. Use the supplied or agreed implementation. If it implements a subset or uses decode-then-operate, name that scope and interpret timings accordingly. Full reimplementation of every format is not assumed. The four shared baselines plus the assigned additional format should receive the same logical inputs.

Optional references

Sources and scope of the explanations

The references identify the real formats behind this handout. Reading them is **not required** to use the handout. All worked inputs, calculations, AND traces and proposed experiments here were constructed for this assignment; they are not measured results from these papers.

[1] PLWAH

François Deliège and Torben Bach Pedersen. **Position List Word Aligned Hybrid: Optimizing Space and Performance for Compressed Bitmaps.** EDBT, 2010, pp. 228–239. Sections 2 and 4 describe the encoding and operations. The handout uses the 32-bit, single-position, fixed-counter configuration, without the adaptive counter extension.

Open paper · DOI: 10.1145/1739041.1739071

[2] CONCISE

Alessandro Colantonio and Roberto Di Pietro. **CONCISE: Compressed ’n’ Composable Integer Set.** Information Processing Letters 110(16), 2010, pp. 644–650. Section 2 of the open preprint specifies the flag convention, flipped-bit field and group-count-minus-one field used here.

Open preprint · DOI: 10.1016/j.ipl.2010.05.018

[3] VAL-WAH

Gheorghi Guzun, Guadalupe Canahuate, David Chiu and Jason Sawin. **A Tunable Compression Framework for Bitmap Indices.** ICDE, 2014, pp. 484–495. Sections III–IV describe legal segment lengths, packed flags, selection and conversion during queries. The examples use w = 64 and alignment factor b = 16.

Author's paper · DOI: 10.1109/ICDE.2014.6816675

[4, 5] SBH

[4] Sangchul Kim, Junhee Lee, Srinivasa Rao Satti and Bongki Moon. **SBH: Super byte-aligned hybrid bitmap compression.** Information Systems 62, 2016, pp. 155–168.

Publication

[5] Sangchul Kim. **An Improved Bitmap Index Compression Scheme.** Master's thesis, Seoul National University, 2015. Chapter 3 provides the byte rules and expansion procedure used in the examples. The handout's two-buffer AND trace is an explanatory baseline, not a reproduction of the optimized benchmark implementation.

Open thesis

[6] BBC

Gennady Antoshenkov. **Byte aligned data compression.** Patent publication EP0650264A1, 1995. The description's short-gap case gives the three-field control layout illustrated here. Its other control cases and stream framing must be considered for a full implementation.

Patent description